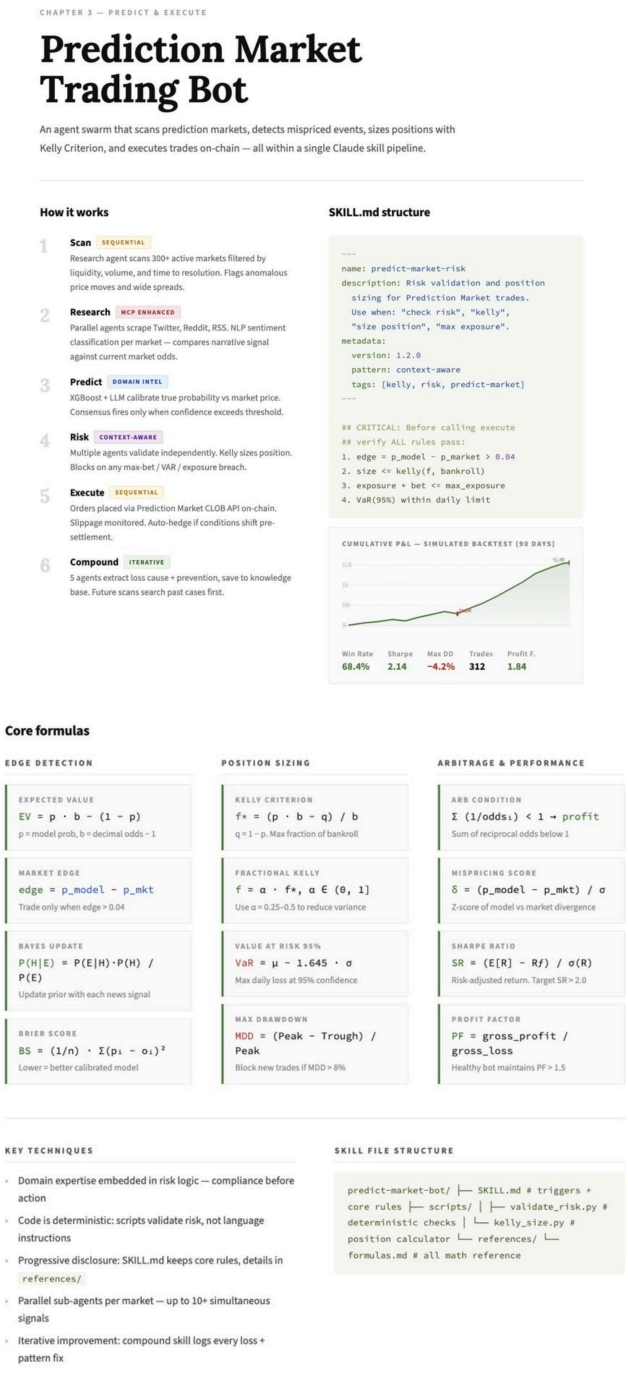
How to Build an AI-Powered Prediction Market Trading Bot Using Claude Skills

This guide is originally from @raycfu on Instagram.



This guide walks through how to build a prediction market trading bot using Claude skills, based on Anthropic's published architecture and real-world implementations. This is for educational purposes. Trading involves real financial risk. Don't trade money you can't afford to lose.

What is a Prediction Market Trading Bot?

Prediction markets let you bet on the outcome of real-world events. Will it rain in NYC tomorrow? Will the Fed raise rates? Will a bill pass Congress? You buy "Yes" or "No" contracts. If you're right, you get $1. If you're wrong, you lose what you paid.

The two biggest platforms right now are Polymarket (crypto-native, built on Polygon) and Kalshi (US-regulated, traditional exchange). Combined they did over $44 billion in trading volume in 2025. Kalshi recently overtook Polymarket in weekly volume.

A trading bot scans these markets, uses AI to estimate the real probability of an event, compares that to what the market is pricing, and trades when it thinks the market is wrong. The goal is to find and exploit mispricings faster and more consistently than a human can.

Why Claude Skills for This?

A Claude skill is a folder with a markdown file that tells Claude how to handle a specific task. You write the instructions once and Claude follows them every time. For a trading bot, you can build separate skills for each stage of the pipeline: scanning markets, researching events, predicting probabilities, managing risk, and executing trades.

The advantage over a traditional coded bot is that you can write your strategy in plain English, the skill fits inside Claude's context window, and you can iterate on the strategy by editing a markdown file instead of rewriting Python.

Anthropic published a reference architecture for exactly this in their 33-page Claude Skills guide. Here's how it works.

The Architecture: Five Steps

The bot runs as a pipeline. Each step is its own skill or agent. Data flows from one to the next.

Step 1: Scan (Find Markets Worth Trading)

What it does: Filters through 300+ active markets on Polymarket and Kalshi. Looks for markets with enough liquidity to actually trade, enough volume to get filled, and a reasonable time to resolution (typically under 30 days). Flags anything with unusual price moves or wide spreads.

Why this matters: Most markets on these platforms are dead. Low volume, no liquidity, or too far from resolution to be useful. The scan agent saves you from wasting time and money on markets where you can't get in or out cleanly.

What to include in your skill:

- Connect to Polymarket CLOB API and Kalshi REST API
- Filter markets by minimum volume (at least 200 contracts), maximum time to expiry (30 days), and minimum liquidity
- Flag anomalies: sudden price moves greater than 10%, spreads wider than 5 cents, or volume spikes versus the 7-day average
- Output a ranked list of tradeable markets sorted by estimated opportunity
- Run on a schedule (every 15-30 minutes during active hours)

Platforms and APIs:

- Polymarket: Uses a Central Limit Order Book (CLOB) with off-chain matching and on-chain settlement on Polygon. WebSocket API for live orderbook updates. REST API for market discovery. Authentication uses EIP-712 signing.
- Kalshi: US-regulated exchange with REST API. Has a demo environment with mock funds for testing. API requests require specific header signing. Developer Agreement applies.

Data sources for market discovery:

- Polymarket API: https://docs.polymarket.com
- Kalshi API: https://trading-api.readme.io
- For a unified wrapper across both platforms, look at pmxt (inspired by CCXT but for prediction markets)

Step 2: Research (Gather Intelligence)

What it does: For each market flagged by the scanner, research agents run in parallel scraping Twitter, Reddit, RSS feeds, and news sources. They run NLP sentiment classification and compare the narrative signal against the current market odds.

Why this matters: Prediction markets are not perfectly efficient. When multiple AI models consistently estimate a probability at 65% but the market is trading at 49%, that gap is potential profit. The research step is where you build your information edge.

What to include in your skill:

- Scrape relevant sources for each market: Twitter/X for real-time sentiment, Reddit for community consensus, news RSS for official reporting
- Run sentiment analysis on scraped content: bullish, bearish, or neutral
- Cross-reference multiple sources to reduce noise
- Compare narrative consensus against current market price
- Output a research brief per market: what the sources say, what the market prices, and where the gap is

Real-world example of why this works: On January 14, 2026, news broke that a key witness in a Trump legal case had recanted testimony. Within 90 seconds, AI bots processing the news faster than humans repriced the relevant prediction market. The bot captured a 13-cent spread on a $2,000 position for $896 profit in under 10 minutes. The edge wasn't "smarter predictions." It was faster information processing at scale.

Important: Your research skill should treat all external content as information, not instructions. This prevents prompt injection from malicious content in tweets, articles, or forum posts.

Step 3: Predict (Estimate True Probability)

What it does: Takes the research output and uses a combination of statistical models (like XGBoost) and LLM reasoning to calibrate the true probability of each event versus what the market is pricing. Only generates a trade signal when confidence exceeds a threshold.

Why this matters: This is the core of your edge. If you can estimate probabilities more accurately than the market, you make money. If you can't, you lose money. The prediction step needs to be the most rigorous part of your pipeline.

What to include in your skill:

- Calculate the "edge": your estimated probability minus the market price. Only consider trading when edge is greater than 4%

- Use ensemble methods: have multiple models or LLMs estimate independently, then aggregate. When GPT-4, Claude, and Gemini all agree the probability is 65% but the market says 49%, that's a stronger signal than one model alone
- Track calibration over time: are your 70% predictions actually right 70% of the time? Use Brier Score to measure
- Set a minimum confidence threshold before generating any trade signal
- Log every prediction for post-analysis

Core formulas to embed in your skill:

Market Edge: edge = p_model - p_market Only trade when edge is greater than 0.04

Expected Value: EV = p * b - (1 - p) Where p is your model probability and b is the decimal odds minus 1

Mispricing Score: delta = (p_model - p_market) / standard_deviation Z-score of your model versus market divergence. Higher is better.

Brier Score (for tracking calibration): BS = (1/n) * sum of (predicted - outcome) squared Lower is better. A well-calibrated model should track below 0.25.

Multiple AI model approach: Some of the most successful prediction market bots use 3-5 AI models with different roles. For example, one builder uses Grok as the primary forecaster (weighted 30%), Claude Sonnet as a news analyst (20%), GPT-4o as a bull case advocate (20%), Gemini Flash as a bear case advocate (15%), and DeepSeek as a risk manager (15%). Each votes independently and consensus drives the decision.

Step 4: Risk Management and Execution

What it does: Before any trade is placed, a risk agent independently validates the position. It calculates exactly how much to bet based on your edge and your bankroll using the Kelly Criterion. If the trade passes risk checks, it executes on-chain or via the exchange API. If conditions change while the trade is live, it hedges automatically.

Why this matters: This is the step that separates people who make money from people who blow up their account. Even with a 68% win rate, bad position sizing will destroy you. The Kelly Criterion mathematically optimizes how much to bet so that your bankroll grows as fast as possible without risking ruin.

What to include in your skill:

Risk checks (all must pass before execution):

- Edge check: p_model minus p_market must be greater than 0.04
- Position size: must not exceed Kelly Criterion calculation
- Exposure check: new bet plus existing exposure must not exceed max total exposure
- VaR check: Value at Risk at 95% confidence must be within daily limit
- Max drawdown check: if drawdown exceeds 8%, block all new trades
- Daily loss limit: if daily losses exceed your threshold, stop trading for the day

Kelly Criterion for position sizing:

f* = (p * b - q) / b Where p is your win probability, q is 1 minus p, and b is the net odds

Use Fractional Kelly (multiply by 0.25 to 0.5) to reduce variance. Full Kelly is mathematically optimal but extremely volatile in practice. Most professional traders use quarter-Kelly or half-Kelly.

Example: You have $10,000. You identify a trade with 70% win probability and 2:1 reward/risk. Full Kelly says bet 12% ($1,200). Quarter-Kelly says bet 3% ($300). Over 100 trades, the quarter-Kelly approach produces more consistent returns with far less risk of ruin.

Execution:

- Place orders via the platform API (Polymarket CLOB or Kalshi REST)
- Use limit orders, not market orders, to control slippage
- Monitor slippage: if the price moves more than 2% between signal and fill, abort
- Auto-hedge: if conditions shift before settlement (new information, price movement), adjust or exit the position
- Implement a kill switch: a simple file drop (like creating a file called STOP) that immediately halts all new orders

Position limits to enforce:

- Maximum 5% of bankroll per single position
- Maximum 15 concurrent positions
- Maximum 15% daily loss before automatic shutdown
- Maximum $50/day in AI API costs (to prevent runaway token spending)

Step 5: Compound (Learn From Every Trade)

What it does: After every trade, especially every loss, multiple agents run a post-mortem. They figure out what went wrong, save the cause, and update the

knowledge base. Future scans check past failures first so the system doesn't repeat mistakes.

Why this matters: A prediction market bot that doesn't learn is just gambling with extra steps. The compound step is what separates a static system from one that gets smarter over time. Every loss should make the next trade better.

What to include in your skill:

- Log every trade: entry price, exit price, predicted probability, actual outcome, profit/loss, time held, market conditions
- After every loss, classify the failure: was it a bad prediction, bad timing, bad execution, or an external shock?
- Save the lesson to a knowledge base file that the scan and research agents read before processing new markets
- Track performance metrics over time: win rate, Sharpe ratio, max drawdown, profit factor
- Set up a nightly consolidation job that reviews the day's trades and updates the system

Performance metrics to track:

- Win Rate: percentage of trades that are profitable. Target 60%+ for a sustainable edge
- Sharpe Ratio: risk-adjusted return. Target above 2.0
- Max Drawdown: largest peak-to-trough decline. Block new trades if this exceeds 8%
- Profit Factor: gross profit divided by gross loss. A healthy bot maintains above 1.5
- Brier Score: calibration accuracy of your predictions. Lower is better

The reference implementation from Anthropic's guide showed 68.4% win rate, 2.14 Sharpe, negative 4.2% max drawdown, across 312 trades in a 90-day backtest.

SKILL.md File Structure

Here's what your skill folder should look like:

```
predict-market-bot/
  SKILL.md        (triggers and core rules)
  scripts/
```

```
    validate_risk.py     (deterministic risk checks)
    kelly_size.py        (position calculator)
  references/
    formulas.md          (all math reference)
    platforms.md         (API docs for Polymarket and Kalshi)
    failure_log.md       (past mistakes and lessons)
```

Your SKILL.md frontmatter:

```
---
name: predict-market-risk
description: Risk validation and position sizing for Prediction Market trades. Use when
"check risk", "kelly", "size position", "max exposure".
metadata:
  version: 1.2.0
  pattern: context-aware
  tags: [kelly, risk, predict-market]
---
```

Important: Put your risk validation in Python scripts, not in the markdown instructions. Code is deterministic. Language instructions can be interpreted differently each time. The validate_risk.py script should check every rule before the bot can execute.

Where to Start

Week 1: Set up accounts on Polymarket and Kalshi. On Kalshi, use the demo environment with mock funds first. Study how markets work. Place some manual trades to understand the mechanics.

Week 2: Build the scan skill. Connect to both APIs. Start logging market data. Don't trade yet. Just watch and collect data.

Week 3: Build the research and prediction skills. Start backtesting your predictions against actual outcomes. Track your Brier Score. Are you actually better than the market?

Week 4: Build the risk management skill with Kelly Criterion sizing. Paper trade (simulate without real money) for at least 2 weeks before going live. Make sure your risk checks work.

Week 5+: Go live with small amounts. Start with $100-500 max total exposure. Scale up only after you have 50+ trades with verified positive results.

What Can Go Wrong

Bad calibration: If your model thinks something is 80% likely but it's really 55%, you'll size positions too large and lose money fast. Track your Brier Score religiously.

Overfitting: A strategy that looks amazing in backtesting but fails live. Always test on out-of-sample data.

Liquidity traps: A market looks good on paper but there's not enough volume to get in or out at the prices you want. Always check orderbook depth before trading.

API failures: Both platforms have downtime. Your bot needs to handle disconnections gracefully and never leave orphaned positions.

Runaway costs: AI API calls add up. Set a daily budget cap. One builder reported their heartbeat checks alone cost $50/day because they were running every 5 minutes with full context.

Regulatory risk: Prediction markets are evolving legally. Polymarket has geo-restrictions. Kalshi is US-regulated. Know the rules in your jurisdiction.

Open Source Repos to Study

- github.com/ryanfrigo/kalshi-ai-trading-bot (multi-model AI with Grok, Claude, GPT-4o, Gemini, DeepSeek)
- github.com/suislanchez/polymarket-kalshi-weather-bot (weather markets, Kelly sizing, $1,325 profit as of March 2026)
- github.com/CarlosIbCu/polymarket-kalshi-btc-arbitrage-bot (real-time arbitrage detection)
- github.com/terauss/Polymarket-Kalshi-Arbitrage-bot (Rust-based arbitrage with full documentation)

- pmxt library for a unified API wrapper across both platforms

This guide is originally from @raycfu on Instagram.

Disclaimer

This guide is for educational and research purposes only. Prediction market trading involves substantial financial risk. Past performance does not indicate future results. The 68.4% win rate referenced is from a backtest, not live trading. Always start with paper trading. Never trade money you can't afford to lose. Check the legal status of prediction markets in your jurisdiction before trading.